Sprint 11 Full Structured Notes: CSS Frameworks and Preprocessors

0. Where You Are Before Sprint 11

Before Sprint 11, you should already understand these React ideas:

- React is a JavaScript library for building user interfaces.
- Vite helps you create and run React projects.
- JSX lets you write UI-like syntax inside JavaScript.
- Components are reusable pieces of UI.
- Props pass data from parent components to child components.
- `children` lets one component wrap other JSX.
- Conditional rendering lets React show different UI based on data.
- `map()` renders lists.
- Stable keys help React track list items.
- Events let React respond to clicks, typing, and form submissions.
- Controlled inputs store form values in React state.
- `useState` lets React remember changing data.
- State updates should be immutable.
- `useReducer` can organize related state logic.
- `useRef` can access DOM elements or store values that should not cause re-renders.
- `useEffect` runs side effects after React renders.
- Cleanup functions stop old timers, listeners, and subscriptions.
- Custom hooks let you reuse hook logic.
- Context API lets you share app-level values like user, theme, or language.
- Data fetching lets React communicate with APIs or backend services.
- Loading, data, and error states make async UI clear.
- `useMemo`, `useCallback`, and `React.memo` help reduce unnecessary work in some cases.
- React Router connects URLs to React components.
- Dynamic routes, nested routes, 404 routes, protected routes, and lazy-loaded routes help make React apps feel like multi-page applications.
- Sprint 10 introduced shared state, prop drilling, single source of truth, Context, Redux, Zustand, and React DevTools.

So far, you have mostly focused on this question:

> How do I make a React app work correctly?

Sprint 11 focuses on a new question:

> How do I make a React app look consistent and stay easy to style?

---

1. Sprint 11 Goal

The goal of Sprint 11 is to learn how to choose and use a styling system for React apps.

By the end of Sprint 11, you should be able to:

- Explain what a CSS framework is.
- Explain why styling becomes harder as an app grows.
- Explain the difference between utility-first frameworks and component-based frameworks.
- Understand Tailwind-style utility classes at a beginner level.
- Understand Bootstrap-style component classes at a beginner level.
- Explain the advantages and disadvantages of CSS frameworks.
- Explain what a CSS preprocessor is.
- Understand Sass and SCSS at a beginner level.
- Use SCSS variables.
- Use SCSS nesting.
- Use SCSS mixins.
- Understand the difference between SCSS syntax and indented Sass syntax.
- Use `className` correctly in React JSX.
- Fix common beginner mistakes in Sass, SCSS, and React styling.
- Build a simple styled React project such as Pricing Cards.
- Choose a styling approach intentionally instead of randomly.

Simple version:

> Sprint 11 teaches you how to style React apps in a cleaner, more consistent, and more maintainable way.

---

2. Big Mental Model

A React app has two major sides:

```
App behavior
App appearance
```

Earlier sprints focused mostly on app behavior:

```
State
Events
Forms
Effects
Fetching
Routing
```

```
Shared state
Debugging
```

Sprint 11 focuses on app appearance:

```
Layout
Spacing
Colors
Buttons
Cards
Forms
Responsive design
Reusable style rules
Consistent visual system
```

A beginner React app can use simple CSS.

A larger React app needs a styling plan.

Big Sprint 11 idea:

> Styling is not only about making things look nice. Styling is also about consistency, reuse, maintainability, and team readability.

---

3. Why Styling Becomes a Problem

In a very small app, this kind of CSS is fine:

```css
.card {
  border: 1px solid gray;
  padding: 16px;
}
```

But a larger app may have many UI pieces:

```
Navbar
Sidebar
Dashboard cards
Forms
Buttons
Modals
Pricing cards
```

```
Product cards
Course cards
Error messages
Loading states
Footer
Profile page
Settings page
Login page
```

If every component creates styles randomly, the app starts to feel inconsistent.

Example problem:

```css
.card {
  padding: 12px;
}

.product-card {
  padding: 20px;
}

.course-box {
  padding: 14px;
}
```

These styles may all work, but there is no design system.

The app may look messy because spacing, colors, borders, and font sizes are not consistent.

Beginner rule:

> When the app grows, styling needs a plan.

---

4. Styling Strategy

A styling strategy is the way you decide to write and organize styles.

Common styling strategies include:

```
Plain CSS
CSS Modules
SCSS/Sass
Tailwind CSS
```

```
Bootstrap
Component libraries
Design systems
A controlled mix of tools
```

Sprint 11 focuses mainly on:

```
CSS frameworks
CSS preprocessors
```

CSS frameworks help you use ready-made styling tools.

CSS preprocessors help you write more organized custom CSS.

Part One: CSS Frameworks

---

5. What Is a CSS Framework?

A CSS framework is a collection of ready-made CSS tools.

Simple definition:

> A CSS framework gives you prebuilt styling patterns so you do not have to write every style from zero.

A CSS framework may give you:

- spacing classes
- color classes
- layout helpers
- grid systems
- buttons
- cards
- navbars
- modals
- forms
- responsive helpers
- utility classes
- ready-made components

Common examples:

```
Tailwind CSS
Bootstrap
Bulma
Material UI
Chakra UI
```

Do not worry about mastering all of them now.

Sprint 11 focuses on the idea behind CSS frameworks.

---

6. Why Developers Use CSS Frameworks

Developers use CSS frameworks because they can make styling faster and more consistent.

Frameworks can help with:

- faster development
- consistent spacing
- consistent colors
- consistent buttons
- responsive layouts
- browser compatibility
- less repeated CSS
- common UI patterns

Example:

Without a framework, you may write your own button CSS:

```css
.button {
  padding: 10px 16px;
  border-radius: 6px;
  background-color: blue;
  color: white;
}
```

With a framework, you may use existing classes:

```
<button className="btn btn-primary">Save</button>
```

or utility classes:

```
<button className="px-4 py-2 rounded bg-blue-500 text-white">
  Save
</button>
```

Beginner rule:

> A CSS framework helps you move faster, but it does not replace good design thinking.

---

7. Two Main Types of CSS Frameworks in Sprint 11

Sprint 11 compares two main styling approaches:

```
Utility-first frameworks
Component-based frameworks
```

They solve styling problems differently.

Utility-first frameworks give you small classes.

Component-based frameworks give you larger ready-made UI patterns.

Part Two: Utility-First Frameworks

---

8. What Is a Utility-First Framework?

A utility-first framework gives you many small CSS classes.

Each class usually does one small styling job.

Simple definition:

> A utility-first framework lets you build designs by combining small single-purpose classes.

Tailwind CSS is the most common example.

Example:

```
<button className="bg-blue-500 text-white px-4 py-2 rounded">
  Save
</button>
```

Read it like this:

```
bg-blue-500   -> background color
text-white    -> text color
px-4          -> horizontal padding
py-2          -> vertical padding
rounded       -> rounded corners
```

Each class does a small job.

Together, they create the button design.

---

9. Utility-First Mental Model

Normal custom CSS approach:

```
Create a class name
Write CSS for that class
Use the class in JSX
```

Example:

```
.save-button {
  background-color: blue;
  color: white;
  padding: 8px 16px;
  border-radius: 6px;
}
```

```
<button className="save-button">Save</button>
```

Utility-first approach:

```
Use small existing classes directly in JSX
```

Example:

```
<button className="bg-blue-500 text-white px-4 py-2 rounded">
  Save
</button>
```

Mental model:

> Utility-first styling means you compose styles directly with small classes.

---

10. Utility-First Example: Card

Example:

```
<section className="p-6 rounded-lg shadow">
  <h2 className="text-xl font-bold">Pro Plan</h2>
  <p className="mt-2">For growing projects.</p>
</section>
```

Read it like this:

```
p-6           -> padding
rounded-lg    -> large rounded corners
shadow        -> box shadow
text-xl       -> extra-large text
font-bold     -> bold font
mt-2          -> margin top
```

This creates a styled card without writing a separate `.card` CSS rule.

---

11. Advantages of Utility-First Frameworks

Utility-first frameworks are useful because:

- You can build custom designs quickly.
- You do not need to invent many class names.
- Styles are close to the JSX they affect.
- Spacing and colors can stay consistent.
- You can change styles without jumping between many files.
- It works well for component-based UI.

Example:

```
<article className="p-4 border rounded-lg">
  <h2 className="text-lg font-bold">React Course</h2>
  <p className="text-sm">Learn React step by step.</p>
</article>
```

You can see much of the styling directly from the JSX.

---

12. Disadvantages of Utility-First Frameworks

Utility-first frameworks also have disadvantages.

They can make JSX look crowded:

```
<button className="inline-flex items-center justify-center px-4 py-2 rounded-md
bg-blue-600 text-white font-medium hover:bg-blue-700 disabled:opacity-50">
  Save
</button>
```

This may be hard to read at first.

Other possible disadvantages:

- Lots of classes in JSX.
- Beginners may memorize classes instead of learning CSS fundamentals.
- Similar designs may be repeated across components.
- The markup can become visually noisy.

Beginner rule:

> Utility classes are powerful, but long class strings can become hard to read.

Part Three: Component-Based Frameworks

---

13. What Is a Component-Based CSS Framework?

A component-based CSS framework gives you ready-made UI patterns.

Simple definition:

> A component-based framework gives you larger predefined classes for common UI pieces.

Bootstrap is the classic example.

Example:

```
<div className="card">
  <div className="card-body">
    <h5 className="card-title">Pro Plan</h5>
    <p className="card-text">For growing projects.</p>
    <button className="btn btn-primary">Choose Plan</button>
  </div>
</div>
```

Here, classes like these already have built-in styles:

```
card
card-body
card-title
card-text
btn
btn-primary
```

You are using predesigned UI blocks.

---

14. Component-Based Mental Model

Component-based framework:

```
Use ready-made design blocks.
```

Utility-first framework:

```
Assemble your own design using tiny style classes.
```

With Bootstrap-style frameworks, you usually start with existing component patterns.

Example:

```
<button className="btn btn-success">Submit</button>
<button className="btn btn-danger">Delete</button>
<button className="btn btn-secondary">Cancel</button>
```

The framework already knows what those buttons should look like.

---

15. Advantages of Component-Based Frameworks

Component-based frameworks are useful because:

- They are fast for beginners.
- They give ready-made buttons, cards, modals, and navbars.
- They are good for quick dashboards, admin panels, and prototypes.
- They require less custom CSS at the beginning.
- Their class names are often easy to understand.

Example:

```
<button className="btn btn-primary">Save</button>
```

This is easier to read than a long utility class string.

---

16. Disadvantages of Component-Based Frameworks

Component-based frameworks can also create problems.

Possible disadvantages:

- Apps may look generic.
- It can be harder to create a unique design.
- Overriding default styles can be frustrating.
- The framework may include more CSS than you actually use.
- You may need to follow the framework's expected HTML structure.

Example:

```
<div className="card">
  <div className="card-body">
    ...
```

```
  </div>
</div>
```

Some frameworks expect this structure.

If you change it too much, the design may break or become harder to customize.

Beginner rule:

> Component-based frameworks are fast, but they can make your app look like the framework.

Part Four: Utility-First vs Component-Based

---

17. Utility-First vs Component-Based Comparison

Question	Utility-first	Component-based
Common example	Tailwind CSS	Bootstrap
Main idea	Small classes for tiny style rules	Ready-made UI patterns
Flexibility	High	Medium
Speed	Fast once learned	Very fast at the start
Design uniqueness	Easier to customize	Can look generic
JSX readability	Can get long	Usually shorter
Best for	Custom designs	Quick standard UIs

Simple decision rule:

> Use utility-first when you want more design control.
>
> Use component-based when you want ready-made UI quickly.

---

18. Example: Same Button in Different Styles

Plain CSS approach

```
<button className="save-button">Save</button>
```

```css
.save-button {
  background-color: blue;
  color: white;
  padding: 8px 16px;
  border-radius: 6px;
}
```

Utility-first approach

```jsx
<button className="bg-blue-500 text-white px-4 py-2 rounded">
  Save
</button>
```

Component-based approach

```jsx
<button className="btn btn-primary">Save</button>
```

All three can work.

The important question is:

> Which approach fits the project?

Part Five: CSS Framework Trade-Offs

---

19. Frameworks Are Useful, But Not Automatically Good

A CSS framework is a tool.

A tool is useful only when it solves a real problem.

Do not add a framework only because it is popular.

Ask:

```
Does this framework help this project?
Does the team know it?
Will it make styling faster?
```

```
Will it make the app harder to customize?
Will it add unnecessary CSS?
```

Beginner rule:

> Choose a styling tool intentionally.

---

20. Advantages of CSS Frameworks

CSS frameworks can help you:

- build faster
- keep spacing consistent
- keep colors consistent
- create responsive layouts more easily
- avoid writing repetitive CSS
- use tested cross-browser styles
- build common UI faster
- avoid designing everything from zero

Example:

If your project needs many standard forms and buttons, a framework can save time.

---

21. Disadvantages of CSS Frameworks

CSS frameworks can also cause problems.

Possible problems:

- Too many classes in JSX.
- Styles can be hard to override.
- The site may look generic.
- The CSS bundle may become larger.
- Framework styles may conflict with your custom styles.
- You may depend too much on framework rules.

Beginner rule:

> Frameworks save time, but they also add rules and trade-offs.

---

22. Specificity Issues

Specificity means how strongly a CSS selector applies.

If two CSS rules target the same element, the browser must decide which rule wins.

Example:

```
.button {
  background: green;
}
```

But the framework may have something stronger or loaded later:

```
.btn-primary {
  background: blue;
}
```

Your button may still appear blue instead of green.

Simple definition:

> A specificity issue happens when your style does not apply because another CSS rule wins.

Beginner rule:

> When using a framework, understand that your custom CSS may need to compete with framework CSS.

---

23. Generic Design Problem

Some frameworks have a recognizable look.

If many websites use the same default styles, they can look similar.

Example:

```
Same buttons
Same cards
Same navbar
```

```
Same spacing
Same form controls
```

This is not always bad.

For internal tools or admin dashboards, generic design may be acceptable.

For a brand-heavy public website, it may be a problem.

Beginner rule:

> Ready-made styles are fast, but custom design requires more control.

---

24. Performance Cost

Some CSS frameworks include more CSS than your app actually uses.

This can increase file size.

A larger CSS file can affect loading performance.

Simple idea:

```
More unused CSS -> larger files -> more work for the browser
```

Modern tools can reduce unused CSS in many cases, but you should still be aware of the cost.

Beginner rule:

> Do not import a large styling system if you only need a few small styles.

Part Six: CSS Preprocessors

---

25. What Is a CSS Preprocessor?

A CSS preprocessor lets you write enhanced CSS syntax.

Then it compiles that enhanced syntax into normal CSS.

Simple definition:

> A CSS preprocessor lets you write more powerful CSS, then converts it into regular CSS that browsers understand.

Browsers do not directly understand SCSS features like variables and mixins.

So SCSS must be compiled into normal CSS.

Mental model:

```
You write SCSS
        ↓
Build tool compiles it
        ↓
Browser receives normal CSS
```

---

26. Common CSS Preprocessors

Common CSS preprocessors include:

```
Sass
Less
Stylus
```

Sprint 11 focuses on Sass/SCSS because it is very common.

---

27. What Is Sass?

Sass is a CSS preprocessor.

It adds extra features to CSS, such as:

- variables
- nesting
- modules
- mixins
- inheritance
- operators

You write Sass or SCSS.

The tool compiles it into normal CSS.

Simple definition:

> Sass is a tool that makes CSS easier to organize and reuse.

---

28. What Is SCSS?

SCSS is a syntax of Sass.

It looks very similar to normal CSS.

Example:

```
$primary-color: #3498eb;

.card {
  background-color: $primary-color;
}
```

SCSS uses:

```
braces {}
semicolons ;
CSS-like syntax
```

Beginner recommendation:

> Learn SCSS first because it looks closer to normal CSS.

Part Seven: SCSS Variables

---

29. What Is an SCSS Variable?

An SCSS variable stores a reusable style value.

Simple definition:

> An SCSS variable lets you save a value once and reuse it in many places.

Example:

```
$primary-color: #3498eb;
```

You can use it like this:

```
.button {
  background-color: $primary-color;
}
```

---

30. Why SCSS Variables Are Useful

Without variables, you may repeat the same value many times.

Example without SCSS:

```
.card {
  background-color: #3498eb;
}

.button {
  background-color: #3498eb;
}

.link {
  color: #3498eb;
}
```

If the brand color changes, you must update every copy.

With SCSS:

```
$primary-color: #3498eb;

.card {
  background-color: $primary-color;
}

.button {
  background-color: $primary-color;
}
```

```
.link {
  color: $primary-color;
}
```

Now the color lives in one place.

If you change this:

```
$primary-color: #2563eb;
```

all related styles update.

Mental model:

> SCSS variable = reusable style value.

---

31. Common Values Stored in SCSS Variables

You can use variables for:

```
colors
spacing
font sizes
border radius
box shadows
breakpoints
z-index values
```

Example:

```
$primary-color: #3498eb;
$text-color: #1f2937;
$border-color: #d1d5db;
$spacing: 1rem;
$radius: 12px;
```

Then use them:

```
.card {
  color: $text-color;
```

```
  border: 1px solid $border-color;
  padding: $spacing;
  border-radius: $radius;
}
```

Part Eight: SCSS Nesting

---

32. What Is SCSS Nesting?

SCSS nesting lets you place related CSS rules inside another rule.

Simple definition:

> Nesting lets you write child styles inside the parent style block.

Normal CSS:

```
.card {
  padding: 16px;
}

.card h2 {
  font-size: 24px;
}

.card p {
  color: #555;
}
```

SCSS nesting:

```
.card {
  padding: 16px;

  h2 {
    font-size: 24px;
  }

  p {
    color: #555;
```

```
  }
}
```

This is easier to read when styles belong to the same component.

---

33. Why Nesting Is Useful

Nesting is useful because it keeps related styles together.

Example:

```
.pricing-card {
  padding: 1rem;
  border: 1px solid #ddd;

  h2 {
    margin-top: 0;
  }

  p {
    color: #555;
  }
}
```

You can quickly see that `h2` and `p` styles belong to `.pricing-card`.

---

34. Nesting Warning

Too much nesting is dangerous.

Risky example:

```
.page {
  .section {
    .card {
      .content {
        .title {
          color: red;
        }
      }
    }
```

```
  }
}
```

This creates CSS that is too specific and hard to override.

Good nesting:

```
.card {
  h2 {
    font-size: 24px;
  }
}
```

Beginner rule:

> Nest lightly. Do not create deep CSS chains.

Part Nine: SCSS Mixins

---

35. What Is a Mixin?

A mixin is a reusable group of CSS rules.

Simple definition:

> A mixin lets you define a group of styles once and reuse it in many places.

You define a mixin with `@mixin`.

You use a mixin with `@include`.

---

36. Basic Mixin Example

Define a mixin:

```
@mixin center-flex {
  display: flex;
  justify-content: center;
```

```
  align-items: center;
}
```

Use it:

```
.card {
  @include center-flex;
}
```

Compiled CSS is roughly:

```
.card {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Mental model:

```
@mixin = define reusable style group
@include = use that style group
```

---

37. Why Mixins Are Useful

Mixins are useful when you repeat the same group of CSS rules.

Example repeated CSS:

```
.card {
  border: 1px solid #ddd;
  border-radius: 12px;
  padding: 1rem;
  background: white;
}

.modal {
  border: 1px solid #ddd;
  border-radius: 12px;
  padding: 1rem;
```

```
  background: white;
}
```

With SCSS mixin:

```
@mixin surface-box {
  border: 1px solid #ddd;
  border-radius: 12px;
  padding: 1rem;
  background: white;
}

.card {
  @include surface-box;
}

.modal {
  @include surface-box;
}
```

Now the reusable style group lives in one place.

---

38. Mixin With Variables

Mixins can work with variables.

Example:

```
$border-color: #ddd;
$radius: 12px;
$spacing: 1rem;

@mixin card-surface {
  border: 1px solid $border-color;
  border-radius: $radius;
  padding: $spacing;
  background: white;
}

.pricing-card {
  @include card-surface;
}
```

This combines two SCSS ideas:

```
variables
mixins
```

Part Ten: Sass Syntax vs SCSS Syntax

---

39. Two Sass Syntaxes

Sass has two syntax styles:

```
SCSS syntax
Indented Sass syntax
```

Both are Sass, but they look different.

---

40. SCSS Syntax

SCSS looks like normal CSS.

Example:

```
$primary-color: #3498eb;

.card {
  background-color: $primary-color;
}
```

SCSS uses:

```
braces {}
semicolons ;
```

This is usually easier for beginners because it looks close to CSS.

---

41. Indented Sass Syntax

Indented Sass syntax uses indentation instead of braces and semicolons.

Example:

```
$primary-color: #3498eb

.card
  background-color: $primary-color
```

This is shorter, but it may feel less familiar if you are coming from normal CSS.

Beginner recommendation:

> Use SCSS first.

Part Eleven: React Styling Rule - className

---

42. React Uses `className`, Not `class`

In normal HTML, you write:

```
<section class="card">Hello</section>
```

In React JSX, write:

```
<section className="card">Hello</section>
```

Why?

Because JSX is JavaScript-like syntax, and `class` is a reserved JavaScript word.

Beginner rule:

> In React JSX, always use `className` for CSS classes.

---

43. Common React Styling Mistake

Wrong:

```
export default function App() {
  return <section class="card">Hello</section>;
}
```

Correct:

```
export default function App() {
  return <section className="card">Hello</section>;
}
```

Part Twelve: Fixing the Sprint 11 Debug Code

---

44. Broken Code

Broken code:

```
$primary-color #3498eb;
@mixin center-flex
 display: flex;
 justify-content: center;
<section class="card">Hello</section>
```

This code has several problems.

Problems:

```
$primary-color is missing a colon.
The mixin syntax is incomplete.
The mixin body needs braces in SCSS.
SCSS and JSX are mixed together.
React uses className, not class.
The JSX belongs in a React component file.
The SCSS belongs in a stylesheet file.
```

45. Fixed SCSS

```
$primary-color: #3498eb;

@mixin center-flex {
  display: flex;
  justify-content: center;
  align-items: center;
}

.card {
  @include center-flex;
  background-color: $primary-color;
}
```

What this does:

```
$primary-color stores a reusable color.
@mixin center-flex defines reusable flex centering rules.
@include center-flex uses the mixin inside .card.
.card gets the primary background color.
```

---

46. Fixed React JSX

```
export default function App() {
  return <section className="card">Hello</section>;
}
```

What this does:

```
App returns a section.
The section uses className.
The className connects the JSX element to the .card style in SCSS/CSS.
```

Part Thirteen: Using SCSS in a Vite React App

---

47. Install Sass

In a Vite React project, install Sass with:

```
npm install -D sass
```

Meaning:

```
npm             -> Node package manager
install         -> install a package
-D              -> save as a development dependency
sass            -> the Sass compiler package
```

Why `-D`?

Because Sass is needed during development/build time to compile SCSS into CSS.

---

48. Import an SCSS File

After installing Sass, you can import an SCSS file in React.

Example:

```
import "./App.scss";
```

This tells Vite to process the SCSS file and apply the styles.

---

49. Simple Vite + SCSS Structure

Simple project structure:

```
src/
  App.jsx
  App.scss
```

Example:

```jsx
// src/App.jsx
import "./App.scss";

export default function App() {
  return <h1 className="title">Hello SCSS</h1>;
}
```

```scss
/* src/App.scss */
$title-color: #3498eb;

.title {
  color: $title-color;
}
```

---

50. Better SCSS File Structure

As the project grows, use a more organized structure:

```
src/
  App.jsx
  App.scss
  styles/
    _variables.scss
    _mixins.scss
```

Use `_variables.scss` for reusable values.

Use `_mixins.scss` for reusable style groups.

---

51. Example: Variables File

```scss
/* src/styles/_variables.scss */
$primary-color: #3498eb;
$text-color: #1f2937;
$border-color: #d1d5db;
$spacing: 1rem;
$radius: 12px;
```

This file stores shared design values.

52. Example: Mixins File

```
/* src/styles/_mixins.scss */
@mixin card-surface {
  border: 1px solid #ddd;
  border-radius: 12px;
  padding: 1rem;
  background: white;
}
```

This file stores reusable groups of styles.

---

53. Using Variables and Mixins With `@use`

In modern Sass, use `@use` to load other SCSS files.

Example:

```
@use "./styles/variables" as *;
@use "./styles/mixins" as *;

.card {
  @include card-surface;
  color: $primary-color;
}
```

Meaning:

```
@use "./styles/variables" as *; -> use variables without prefix
@use "./styles/mixins" as *;    -> use mixins without prefix
@include card-surface;           -> apply the mixin
$primary-color                    -> use the variable
```

Part Fourteen: Sprint 11 Example Project - Pricing Cards

---

54. Project Goal

Build a simple Pricing Cards page.

This project practices:

React concepts:

```
components
props
map()
stable keys
spread props
className
semantic HTML
real buttons
```

Sprint 11 styling concepts:

```
SCSS variables
SCSS mixins
SCSS nesting
consistent spacing
consistent card styling
reusable button styling
```

---

55. App.jsx

```jsx
import "./App.scss";

const plans = [
  {
    id: "starter",
    name: "Starter",
    price: "$9",
    features: ["1 project", "Basic support", "Community access"],
  },
  {
    id: "pro",
    name: "Pro",
    price: "$29",
    features: ["10 projects", "Priority support", "Advanced reports"],
```

```
  },
  {
    id: "team",
    name: "Team",
    price: "$79",
    features: ["Unlimited projects", "Team dashboard", "Admin controls"],
  },
];

function PricingCard({ name, price, features }) {
  return (
    <article className="pricing-card">
      <h2>{name}</h2>
      <p className="pricing-card__price">{price}</p>

      <ul>
        {features.map((feature) => (
          <li key={feature}>{feature}</li>
        ))}
      </ul>

      <button type="button" className="pricing-card__button">
        Choose {name}
      </button>
    </article>
  );
}

export default function App() {
  return (
    <main className="pricing-page">
      <h1>Pricing Plans</h1>

      <section className="pricing-grid" aria-label="Pricing plans">
        {plans.map((plan) => (
          <PricingCard key={plan.id} {...plan} />
        ))}
      </section>
    </main>
  );
}
```

---

56. App.jsx Explanation

This imports the SCSS file:

```
import "./App.scss";
```

This array stores the pricing data:

```
const plans = [...];
```

Each plan has:

```
id
name
price
features
```

This component receives props:

```
function PricingCard({ name, price, features }) {
```

This renders a card:

```
<article className="pricing-card">
```

This renders the feature list:

```
{features.map((feature) => (
  <li key={feature}>{feature}</li>
))}
```

This uses a stable key:

```
key={feature}
```

This button uses `type="button"`:

```
<button type="button" className="pricing-card__button">
```

This maps through all plans:

```
{plans.map((plan) => (
  <PricingCard key={plan.id} {...plan} />
))}
```

This uses spread props:

```
{...plan}
```

That passes:

```
name
price
features
```

into `PricingCard`.

---

57. App.scss

```
$primary-color: #3498eb;
$text-color: #1f2937;
$border-color: #d1d5db;
$spacing: 1rem;

@mixin card-surface {
  border: 1px solid $border-color;
  border-radius: 12px;
  padding: $spacing * 1.5;
  background: white;
}

.pricing-page {
  max-width: 960px;
  margin: 0 auto;
  padding: 2rem;

  h1 {
    text-align: center;
    color: $text-color;
  }
}

.pricing-grid {
```

```
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: $spacing;
}

.pricing-card {
  @include card-surface;

  h2 {
    margin-top: 0;
  }

  &__price {
    font-size: 2rem;
    font-weight: bold;
    color: $primary-color;
  }

  &__button {
    width: 100%;
    padding: 0.75rem 1rem;
    border: none;
    border-radius: 8px;
    background: $primary-color;
    color: white;
    cursor: pointer;
  }
}
```

---

58. App.scss Explanation

These are SCSS variables:

```
$primary-color: #3498eb;
$text-color: #1f2937;
$border-color: #d1d5db;
$spacing: 1rem;
```

They store reusable style values.

This is a mixin:

```
@mixin card-surface {
  border: 1px solid $border-color;
  border-radius: 12px;
  padding: $spacing * 1.5;
  background: white;
}
```

It stores reusable card surface styles.

This uses the mixin:

```
.pricing-card {
  @include card-surface;
}
```

This is nesting:

```
.pricing-page {
  h1 {
    text-align: center;
    color: $text-color;
  }
}
```

The `h1` style belongs inside `.pricing-page`.

This is also nesting with `&`:

```
.pricing-card {
  &__price {
    font-size: 2rem;
  }
}
```

`&` means the parent selector.

So this:

```
&__price
```

becomes:

```
.pricing-card__price
```

---

59. What `&` Means in SCSS

In SCSS, `&` means the current parent selector.

Example:

```
.button {
  &--primary {
    background: blue;
  }
}
```

Compiles roughly to:

```
.button--primary {
  background: blue;
}
```

In the Pricing Cards example:

```
.pricing-card {
  &__price {
    color: blue;
  }
}
```

means:

```
.pricing-card__price {
  color: blue;
}
```

Beginner rule:

- `&` refers to the selector you are currently inside.

Part Fifteen: How to Choose a Styling Approach

---

60. Styling Decision Rule

Use this decision rule:

```
Need full control and a small project?
Use plain CSS.

Need reusable variables, mixins, and cleaner custom CSS?
Use SCSS.

Need fast custom UI with utility classes?
Use Tailwind-style utility-first CSS.

Need quick standard UI components?
Use Bootstrap-style component framework.

Need a serious design system?
Use a planned combination, not random mixing.
```

---

61. Recommended Beginner Path

For your current React stage, the best learning path is:

```
1. Understand plain CSS basics.
2. Learn SCSS variables, nesting, and mixins.
3. Compare the same small UI with utility classes.
4. Compare the same small UI with component framework classes.
5. Choose tools based on the project, not popularity.
```

Recommended Sprint 11 focus:

> Learn Sprint 11 with SCSS first, then compare the same small layout with utility-style classes.

Reason:

> SCSS teaches styling fundamentals clearly. Tailwind or Bootstrap will make more sense after you understand the styling problem they solve.

---

62. When to Use Plain CSS

Use plain CSS when:

- the project is small
- you want full control
- you do not need a framework
- your styling needs are simple
- you want to practice CSS fundamentals

Example:

```
.card {
  padding: 1rem;
  border: 1px solid #ddd;
}
```

Plain CSS is not bad.

It is often the simplest correct choice.

---

63. When to Use SCSS

Use SCSS when:

- you want custom CSS
- you want reusable variables
- you want reusable mixins
- you want nesting
- you want cleaner organization than plain CSS
- you are building a custom design

Example:

```
$spacing: 1rem;

.card {
  padding: $spacing;

  h2 {
    margin-top: 0;
  }
}
```

SCSS is a good bridge between plain CSS and larger styling systems.

---

64. When to Use Utility-First CSS

Use utility-first CSS when:

- you want fast styling directly in JSX
- you want a custom design
- your team is comfortable reading utility classes
- you want consistent spacing and color scales
- you want to avoid inventing many class names

Example:

```
<section className="p-6 rounded-lg shadow">
  <h2 className="text-xl font-bold">Pro Plan</h2>
</section>
```

Good for custom UI.

Can become visually noisy if overused carelessly.

---

65. When to Use Component-Based Frameworks

Use component-based frameworks when:

- you need a UI quickly
- you are building a dashboard or admin panel
- you are okay with a standard look
- you want ready-made buttons, cards, forms, and navbars
- design uniqueness is not the main priority

Example:

```
<button className="btn btn-primary">Save</button>
```

Good for speed.

Can be harder to customize deeply.

Part Sixteen: Beginner Practice Tasks

---

66. Warm-Up Drills

Drill 1: Write a utility-style button

Write a button using utility-like classes:

```
<button className="bg-blue-500 text-white px-4 py-2 rounded">
  Save
</button>
```

Explain what each class probably does.

Drill 2: Write an SCSS variable

```
$primary-color: #3498eb;
```

Use it:

```
.button {
  background-color: $primary-color;
}
```

Drill 3: Write a mixin

```
@mixin center-flex {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Use it:

```
.card {
  @include center-flex;
}
```

---

67. Guided Coding Task

Take a previous React project, such as a Course Browser, and style it using SCSS.

Your SCSS should include:

```
variables
nesting
a mixin
consistent spacing
consistent card styles
```

Example variable file:

```
$primary-color: #3498eb;
$text-color: #1f2937;
$spacing: 1rem;
```

Example mixin:

```
@mixin card-surface {
  border: 1px solid #ddd;
  border-radius: 12px;
  padding: 1rem;
  background: white;
}
```

---

68. Debug Task

Fix this broken code:

```
$primary-color #3498eb;
@mixin center-flex
 display: flex;
```

```
 justify-content: center;
<section class="card">Hello</section>
```

Expected fixed SCSS:

```
$primary-color: #3498eb;

@mixin center-flex {
  display: flex;
  justify-content: center;
  align-items: center;
}

.card {
  @include center-flex;
  background-color: $primary-color;
}
```

Expected fixed JSX:

```
export default function App() {
  return <section className="card">Hello</section>;
}
```

---

69. Build Something Small

Build a Pricing Cards page.

Requirements:

- Create an array of pricing plans.
- Render the plans with `map()`.
- Use stable keys.
- Create a reusable `PricingCard` component.
- Use `className`, not `class`.
- Style the layout with SCSS variables.
- Use at least one SCSS mixin.
- Use SCSS nesting.
- Use real buttons.
- Keep spacing consistent.

---

70. Challenge Task

Create a theme variables file.

Example:

```
/* src/styles/_variables.scss */
$primary-color: #3498eb;
$secondary-color: #64748b;
$text-color: #1f2937;
$border-color: #d1d5db;
$spacing-sm: 0.5rem;
$spacing-md: 1rem;
$spacing-lg: 2rem;
$radius-sm: 6px;
$radius-md: 12px;
```

Then use it in your main SCSS file:

```
@use "./styles/variables" as *;

.card {
  border: 1px solid $border-color;
  border-radius: $radius-md;
  padding: $spacing-md;
  color: $text-color;
}
```

Part Seventeen: Common Beginner Mistakes

---

71. Mistake 1: Using `class` Instead of `className`

Wrong:

```
<div class="card">Card</div>
```

Correct:

```
<div className="card">Card</div>
```

React JSX uses `className`.

---

72. Mistake 2: Mixing JSX and SCSS in the Same File Incorrectly

Wrong:

```
.card {
  padding: 1rem;
}

<section className="card">Hello</section>
```

SCSS files should contain styles.

React component files should contain JSX.

Correct file separation:

```
App.jsx     -> React component
App.scss    -> styles
```

---

73. Mistake 3: Forgetting SCSS Semicolons

Wrong:

```
$primary-color: #3498eb
```

Correct:

```
$primary-color: #3498eb;
```

SCSS usually uses semicolons like normal CSS.

---

74. Mistake 4: Wrong Mixin Syntax

Wrong:

```
@mixin center-flex
  display: flex;
```

Correct SCSS:

```
@mixin center-flex {
  display: flex;
}
```

SCSS uses braces for mixin bodies.

---

75. Mistake 5: Over-Nesting SCSS

Risky:

```
.app {
  .page {
    .section {
      .card {
        .title {
          color: red;
        }
      }
    }
  }
}
```

Better:

```
.card-title {
  color: red;
}
```

Beginner rule:

> Keep selectors simple unless deeper nesting is truly needed.

76. Mistake 6: Adding a Framework Without a Reason

Bad reason:

```
I added it because everyone uses it.
```

Better reason:

```
I added it because this dashboard needs many standard forms, buttons, and cards
quickly.
```

Beginner rule:

> Choose tools because they solve project problems.

Part Eighteen: Sprint 11 Revision Checkpoint

---

77. Questions to Answer Without Notes

1. What is a CSS framework?
2. Why does styling become harder as a React app grows?
3. What is a utility-first framework?
4. What is a component-based framework?
5. What is one advantage of utility-first CSS?
6. What is one disadvantage of utility-first CSS?
7. What is one advantage of component-based frameworks?
8. What is one disadvantage of component-based frameworks?
9. What is a specificity issue?
10. What is a CSS preprocessor?
11. What is Sass?
12. What is SCSS?
13. What does an SCSS variable do?
14. What does SCSS nesting do?
15. What is a mixin?
16. What is the difference between `@mixin` and `@include`?
17. Why should beginners usually start with SCSS instead of indented Sass syntax?
18. Why does React use `className` instead of `class`?
19. How do you install Sass in a Vite React project?
20. When should you use plain CSS, SCSS, utility-first CSS, or a component-based framework?

78. Short Answers

1. What is a CSS framework?

A CSS framework gives prebuilt styling tools so you do not have to write every style from zero.

2. Why does styling become harder as a React app grows?

Because many components need consistent spacing, colors, buttons, cards, forms, and layouts.

3. What is a utility-first framework?

A utility-first framework uses small single-purpose classes to build custom designs.

4. What is a component-based framework?

A component-based framework gives ready-made UI patterns like cards, navbars, buttons, and forms.

5. What is one advantage of utility-first CSS?

It gives strong design control and lets you style quickly inside JSX.

6. What is one disadvantage of utility-first CSS?

JSX can become crowded with many classes.

7. What is one advantage of component-based frameworks?

They are very fast for building standard UI.

8. What is one disadvantage of component-based frameworks?

Apps can look generic and may be harder to customize.

9. What is a specificity issue?

A specificity issue happens when your CSS does not apply because another stronger CSS rule wins.

10. What is a CSS preprocessor?

A CSS preprocessor lets you write enhanced CSS syntax and compiles it into normal CSS.

11. What is Sass?

Sass is a CSS preprocessor that adds features like variables, nesting, mixins, modules, inheritance, and operators.

12. What is SCSS?

SCSS is a Sass syntax that looks similar to normal CSS.

13. What does an SCSS variable do?

It stores a reusable style value.

14. What does SCSS nesting do?

It lets you write child styles inside a parent style block.

15. What is a mixin?

A mixin is a reusable group of CSS rules.

16. What is the difference between `@mixin` and `@include`?

`@mixin` defines reusable styles. `@include` uses those styles.

17. Why start with SCSS instead of indented Sass?

SCSS looks closer to normal CSS, so it is easier for beginners.

18. Why does React use `className` instead of `class`?

Because JSX is JavaScript-like syntax, and `class` is a reserved JavaScript word.

19. How do you install Sass in Vite?

```
npm install -D sass
```

20. How do you choose a styling approach?

Use plain CSS for simple projects, SCSS for organized custom CSS, utility-first CSS for fast custom styling, and component-based frameworks for quick standard UI.

Part Nineteen: Definition of Done

---

79. Sprint 11 Is Complete When You Can Explain This

You are done with Sprint 11 when you can explain these points without notes:

```
A CSS framework gives prebuilt styling tools.
Utility-first frameworks use small single-purpose classes.
Component-based frameworks give ready-made UI patterns.
Frameworks are fast but can reduce uniqueness and create override problems.
A preprocessor adds extra CSS syntax and compiles to normal CSS.
Sass is a CSS preprocessor.
SCSS is a Sass syntax that looks like CSS.
SCSS variables store reusable style values.
SCSS nesting groups related styles.
SCSS mixins store reusable groups of style rules.
@mixin defines reusable style rules.
@include uses a mixin.
React uses className, not class.
SCSS and JSX should be kept in the correct file types.
A good styling choice should be intentional, not random.
```

---

80. Final Mental Model

Shortest Sprint 11 mental model:

> Sprint 11 teaches you how to style React apps consistently.

More complete version:

> CSS frameworks help you move faster. Utility-first frameworks give tiny reusable classes. Component-based frameworks give ready-made UI blocks. CSS preprocessors like Sass/SCSS let you write more organized custom CSS with variables, nesting, and mixins. Your job is to choose the styling approach that fits the project instead of randomly adding tools.

---

81. What to Build Before Moving to Sprint 12

Before moving to Sprint 12, build one small styled React project.

Recommended project:

```
Pricing Cards Page
```

It should include:

- one array of pricing plans
- one reusable `PricingCard` component
- `map()` rendering
- stable keys
- `className`
- one SCSS variables section
- one SCSS mixin
- nested SCSS styles
- consistent spacing
- real buttons
- clean file separation between JSX and SCSS

When that works, Sprint 11 is complete.